Architecting for fast flow

Chris Richardson

Founder of Eventuate.io

Founder of the original CloudFoundry.com

Author of POJOs in Action and Microservices Patterns

@crichardson

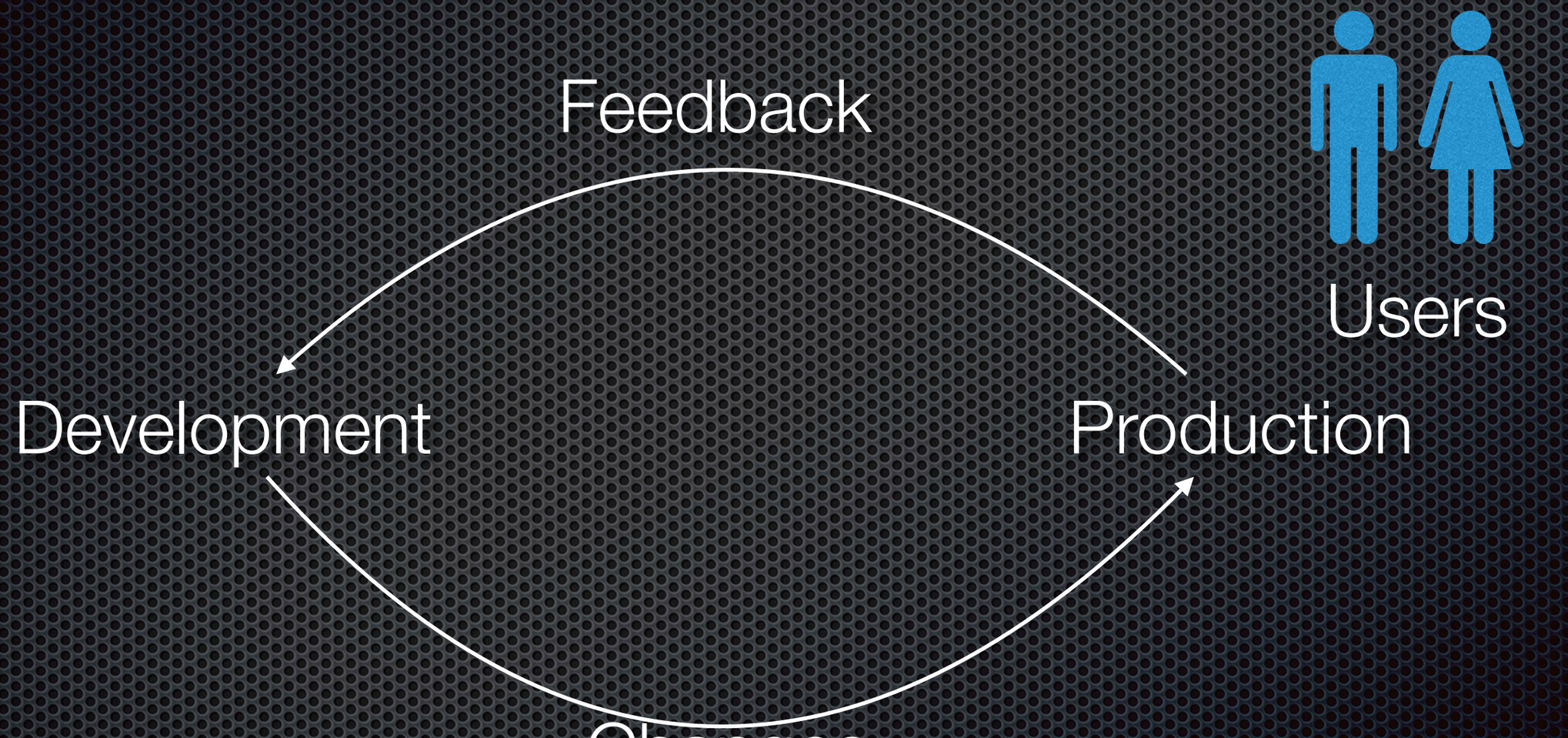
chris@chrisrichardson.net

http://adopt.microservices.io

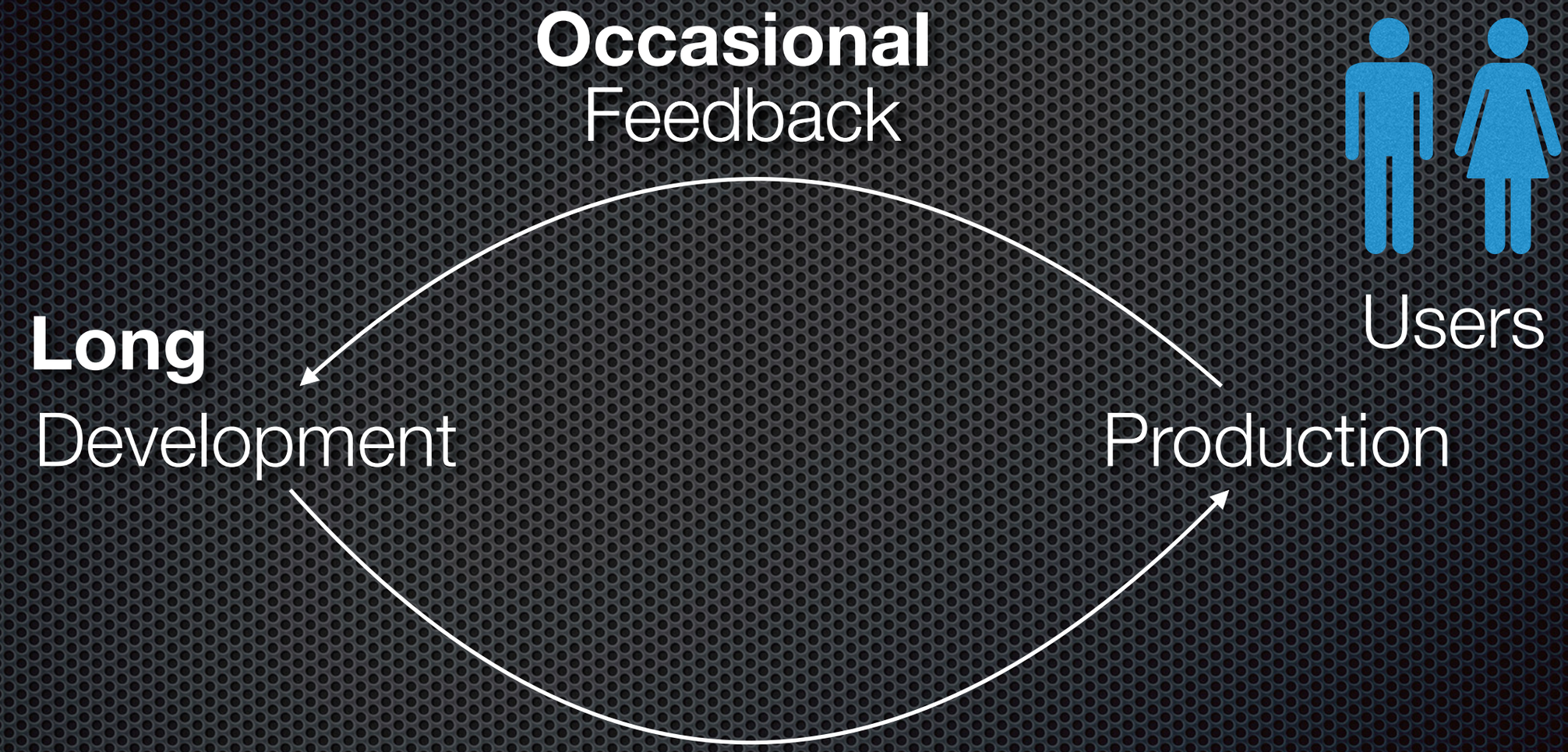
Agenda

- The need for fast flow
- A really brief overview of software architecture
- Architecting for fast flow
- Dark energy forces: encouraging decomposition
- Dark matter forces: resisting decomposition

The software delivery feedback loop



Traditional software delivery, especially outsourced IT projects



Infrequent Changes **e.g. every 1 or 2 years**

Building complex applications

Often using new, unfamiliar technologies

Slow + feedback loops = Build the wrong application, the wrong way

Fast flow for business succe$$

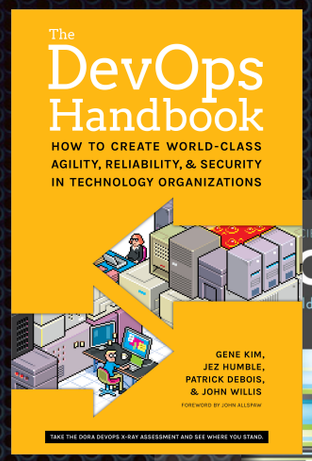
Continuous feedback and learning

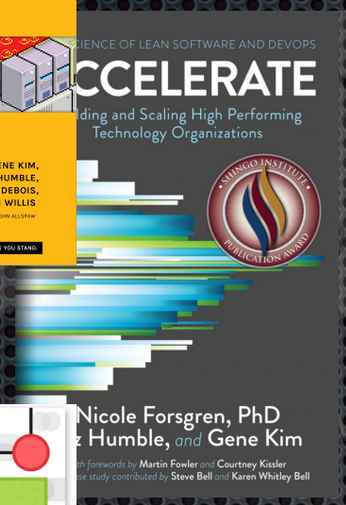
Continuous stream of small changes, many times a day

https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/developer-velocity-how-software-excellence-fuels-business-performance

The success triangle

Process: DevOps/Continuous Delivery & Deployment





S/W VUCA

Businesses must be nimble, agile and innovate faster

IT must continuously deliver a stream of small changes to customers

Enables

Organization:
Network of small,
loosely coupled, product teams

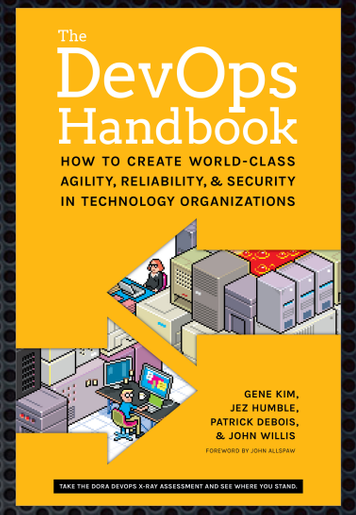
Enables

Architecture:
???

@crichardson

About DevOps

Set of **principles and practices** where developers, testers (dev) and IT operations (ops) collaborate and communicate to deliver software rapidly, frequently, and reliably



Three ways

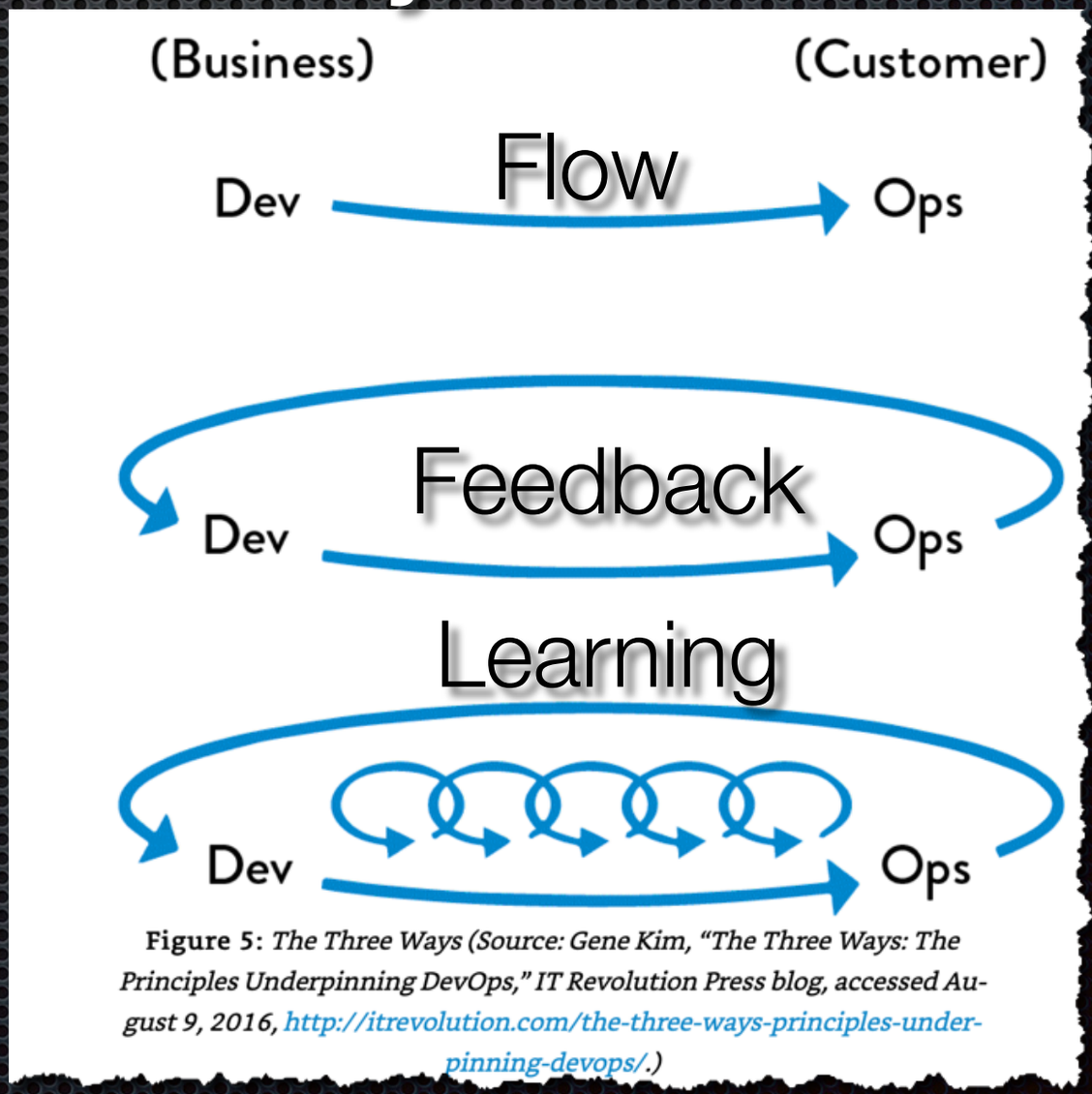


Figure 5: *The Three Ways (Source: Gene Kim, "The Three Ways: The Principles Underpinning DevOps," IT Revolution Press blog, accessed August 9, 2016, http://itrevolution.com/the-three-ways-principles-underpinning-devops/.)*

http://itrevolution.com/devops-handbook

Have a fast deployment pipeline

>= 1 push / developer / day

Must scale to support rate of Git pushes

Developer laptop

Application repository

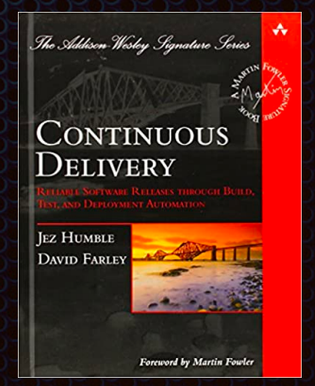
Fast, automated deployment pipeline

Production

15 minutes

Measure using the DORA metrics

https://dora.dev/



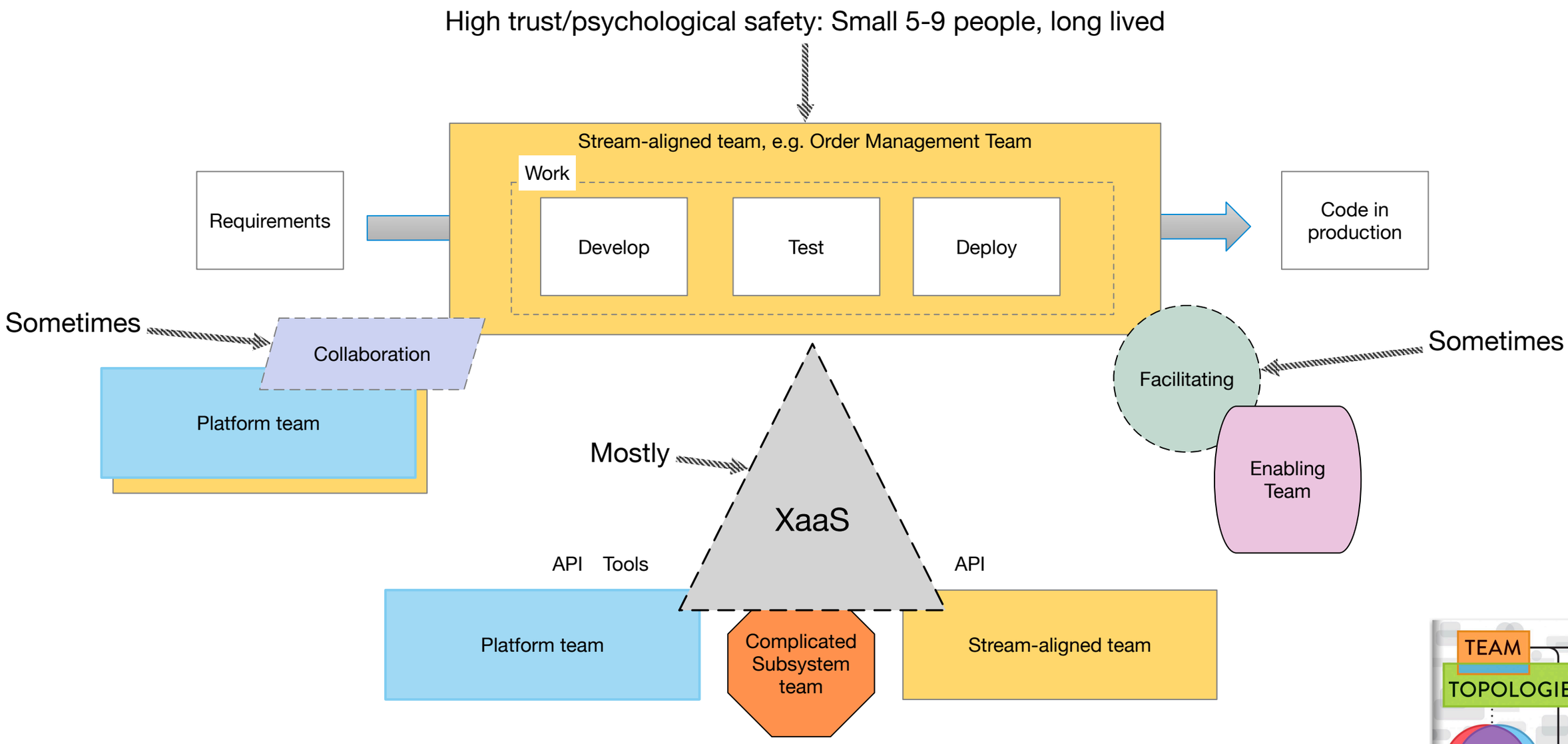
DORA metrics: measuring performance

Metric	Description	Elite performance
Deployment frequency	How frequently changes are pushed to production	Each commit
Lead time	Time from commit to deploy	< 1 hour
Change failure rate	How frequently a change breaks production	Very low
Time to restore service	How long to recover from a failed deployment	Very short

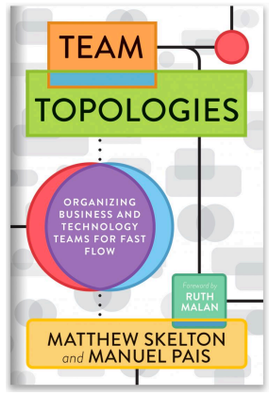
Move fast, don't break things

https://dora.dev/

About Team Topologies: organizing for fast flow



https://teamtopologies.com/key-concepts



“Developer experience (DevEx) encompasses how developers feel about, think about, and value their work.”

https://queue.acm.org/detail.cfm?id=3595878

DevEx framework

Flow

Work environment than minimizes interruptions, meaningful work, …

Fast feedback from colleagues, tools, deployment pipeline, production, users, …

Feedback

Minimize - simplify environment, automate, good documentation, …

Cognitive load

https://queue.acm.org/detail.cfm?id=3595878

Agenda

- The need for fast flow
- A really brief overview of software architecture
- Architecting for fast flow
- Dark energy forces: encouraging decomposition
- Dark matter forces: resisting decomposition

About software architecture

Architecture

"The software architecture of a computing system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both."

Documenting Software Architectures, Bass et al

Satisfies

Focus

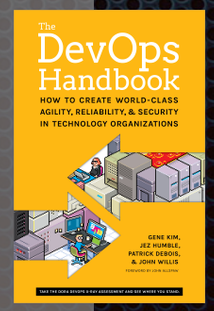
Nonfunctional requirements

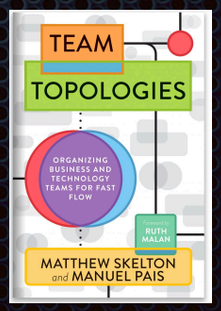
Runtime

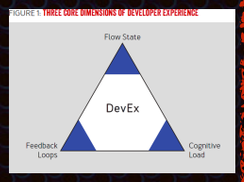
- Scalability
- Performance
- Availability
- …

Development

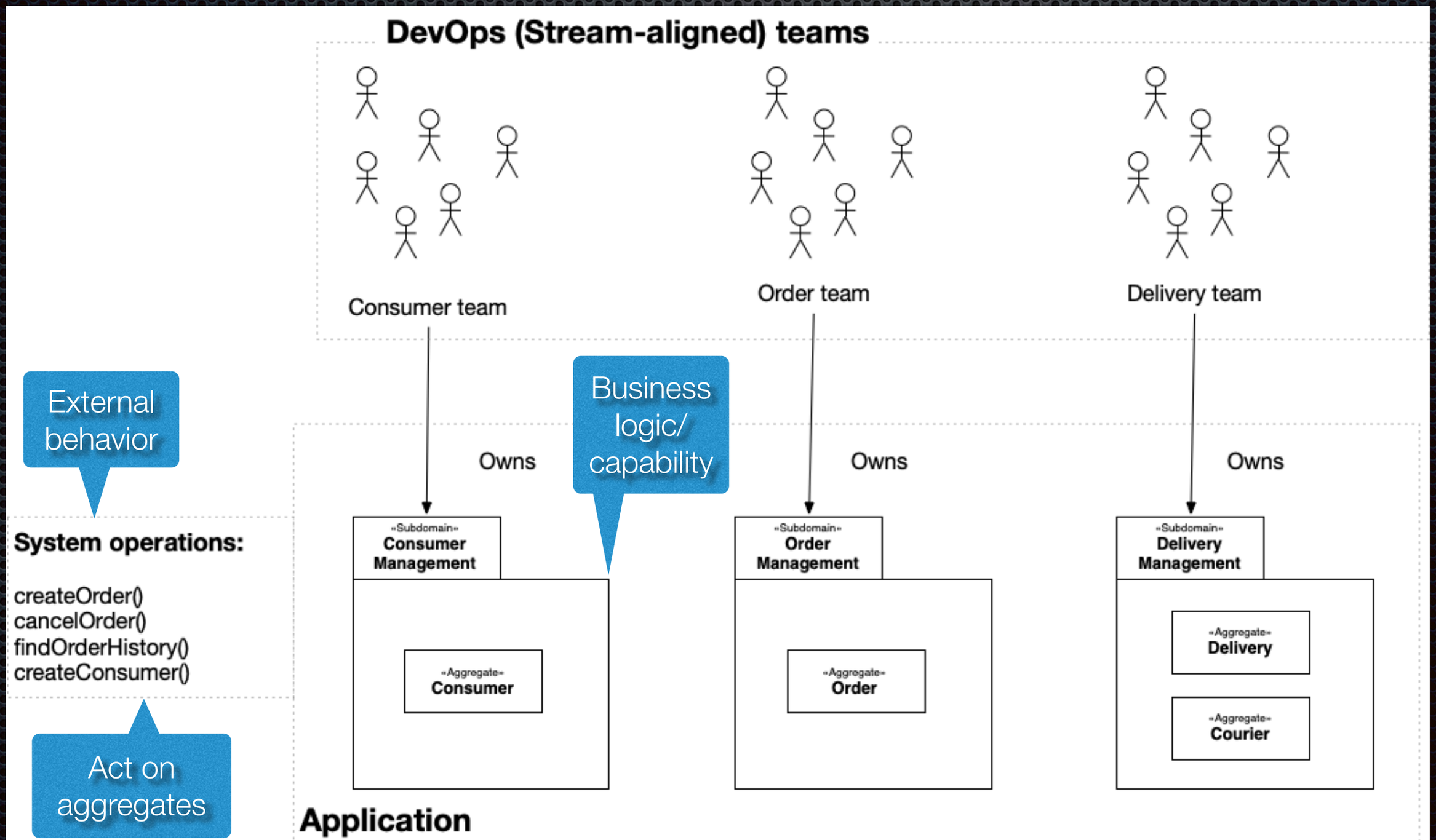
- Maintainability
- Testability
- Deployability
- …



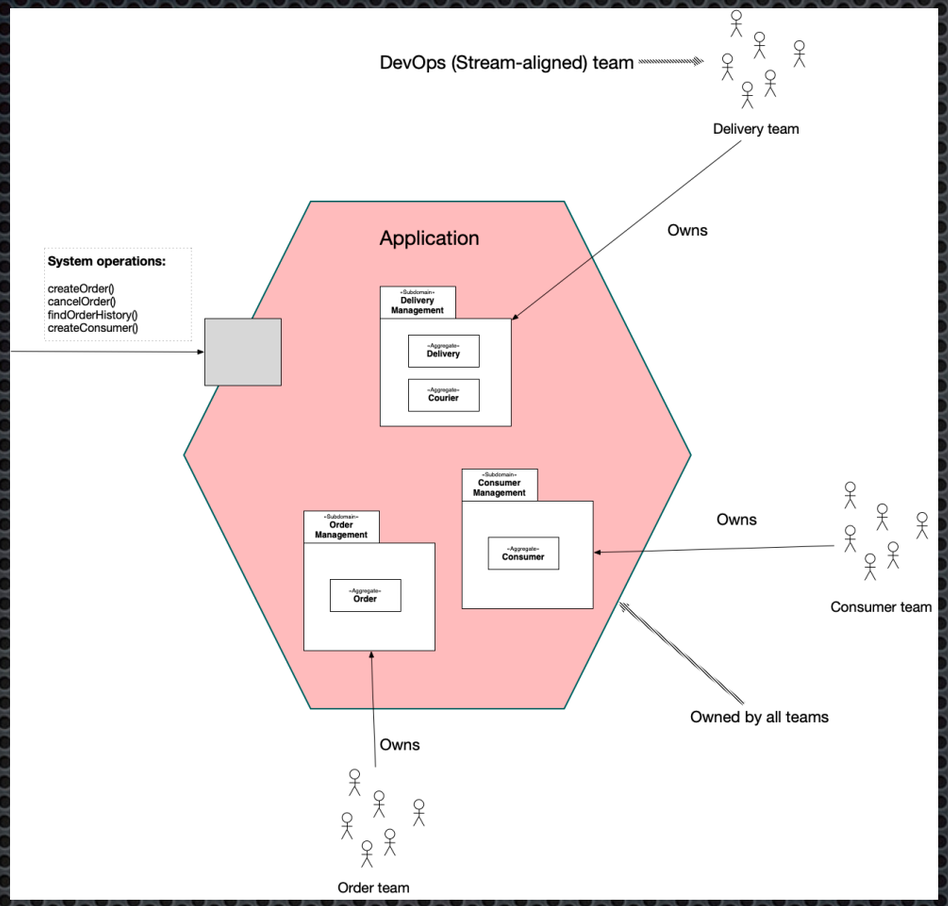




How to organize subdomains into components to meet these requirements?



Monolithic architecture: one component



Developer laptop

Application repository

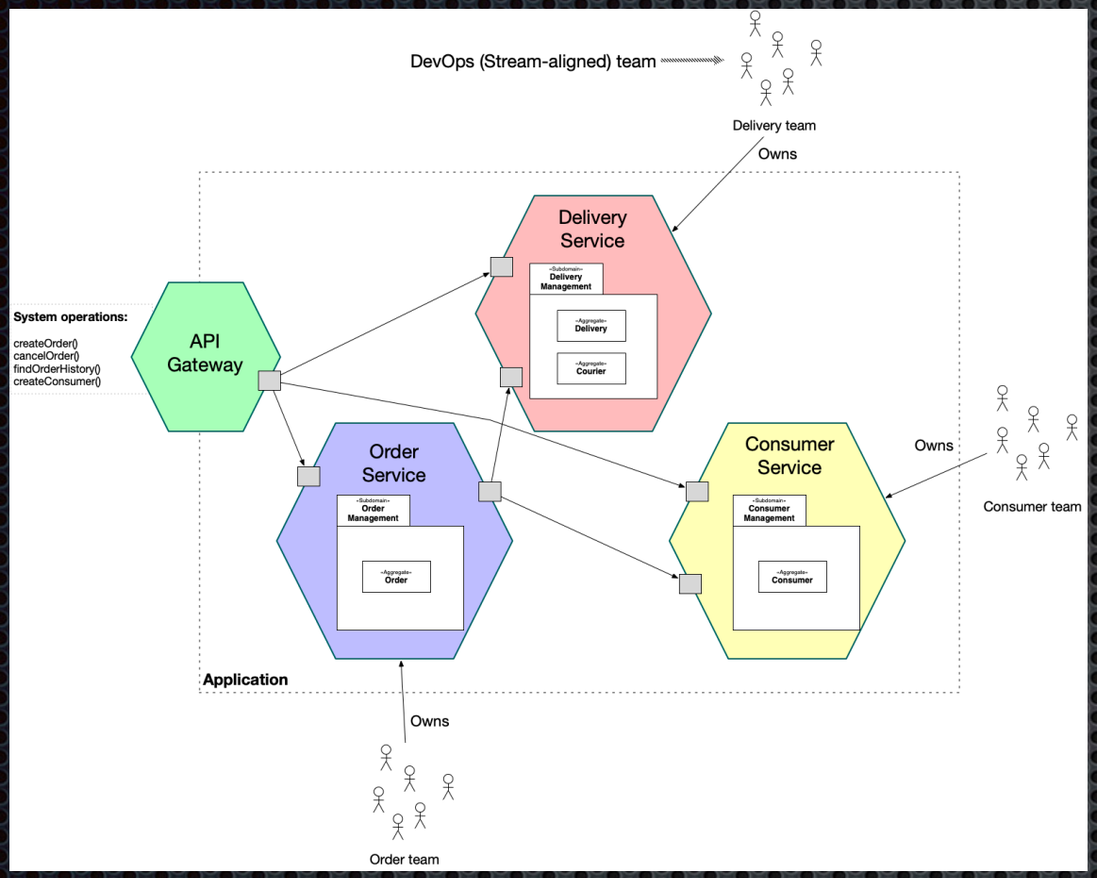
Fast, automated deployment pipeline

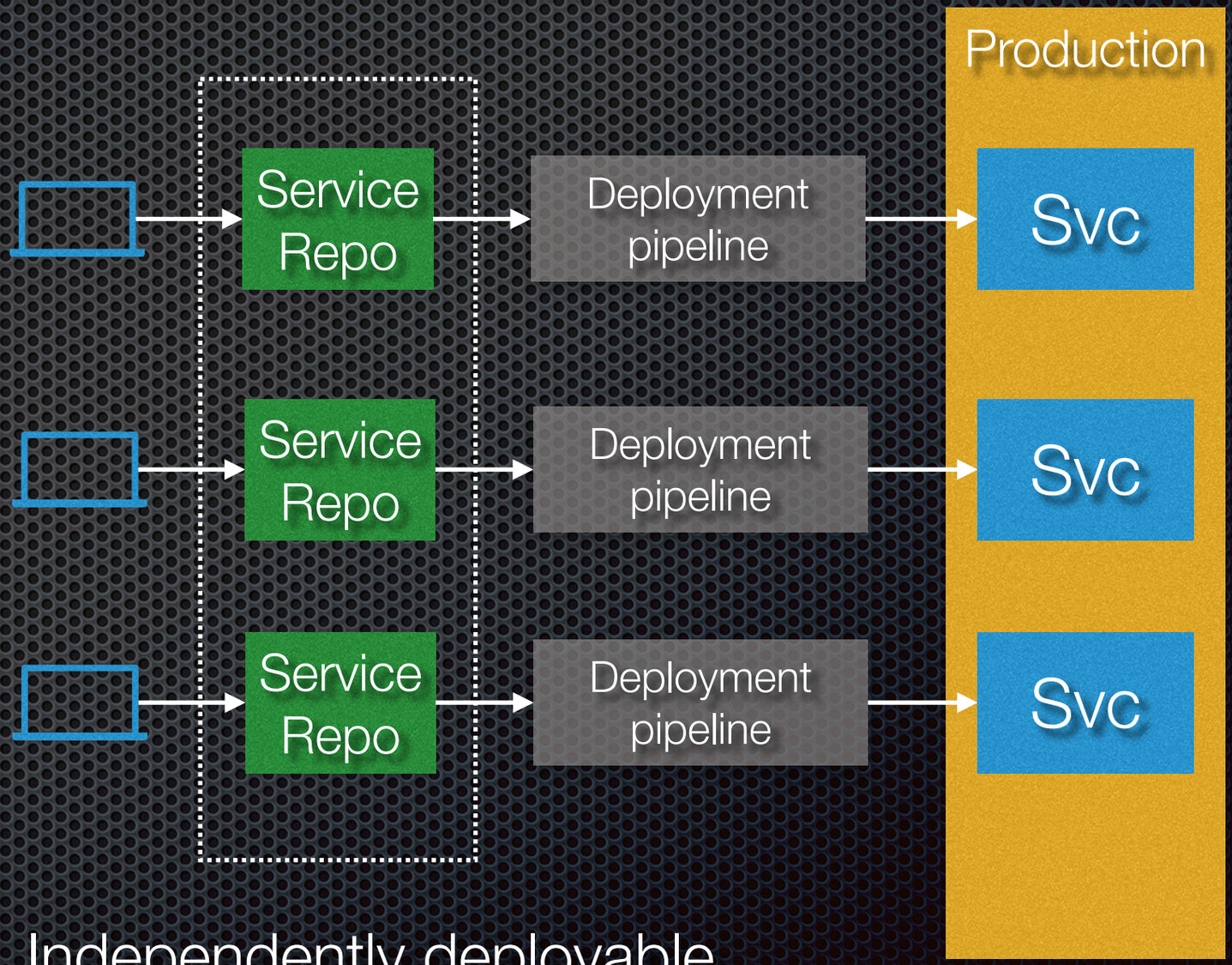
Production

App

@crichardson

Microservice architecture: two or more components



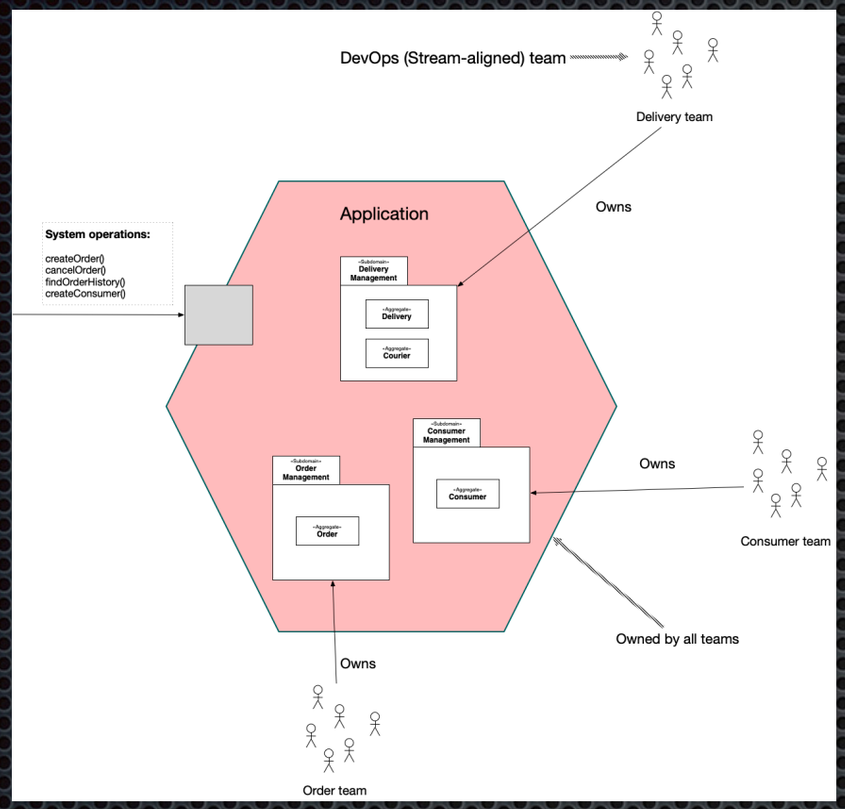


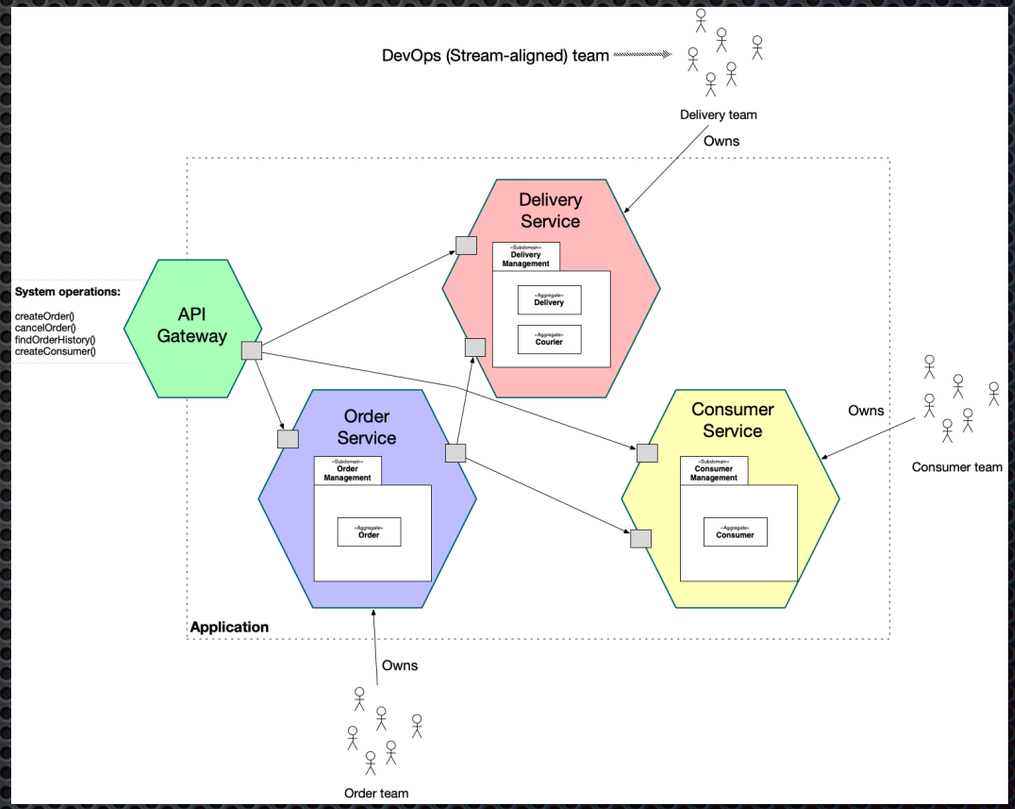
Some system operations span multiple services

Independently deployable

Loosely coupled

Which best fits your application and organization?





#itDepends

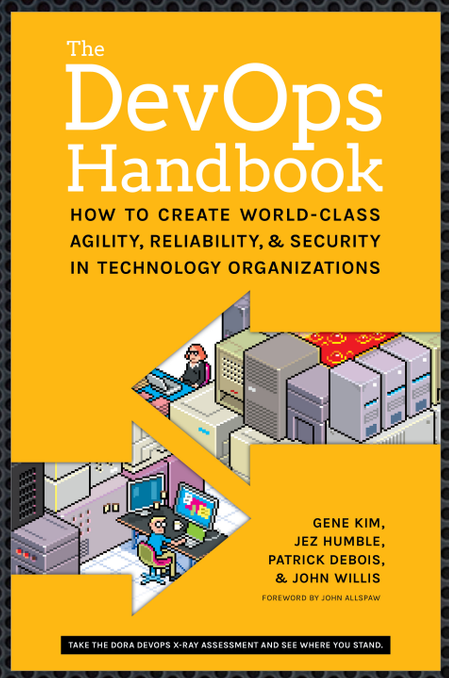
@crichardson

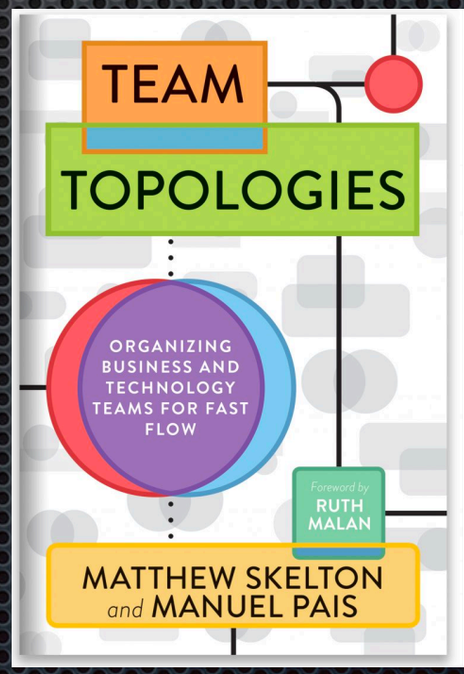
Agenda

- The need for fast flow
- A really brief overview of software architecture
- Architecting for fast flow
- Dark energy forces: encouraging decomposition
- Dark matter forces: resisting decomposition

Monolith vs microservices: which best fits your application/organization?

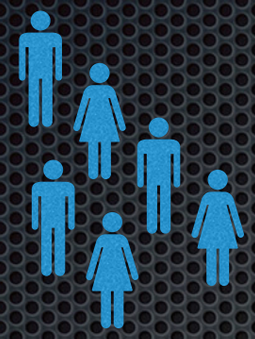
Enables*





Primary reason - there are others

Architectural requirement: loose coupling



← Loosely coupled = work independently →

Conway's law

Owns

Loosely coupled

Unaffected by changes to the other

Owns

Customer Management

Order Management

Degree of design-time coupling = probability of lockstep changes

of classes, packages, modules, services, applications, ...

Loose coupling is a fundamental principle of software design

Ignore at your peril!

Architectural requirement: testable

Handle >= 1 push / developer / day

Must be fast running

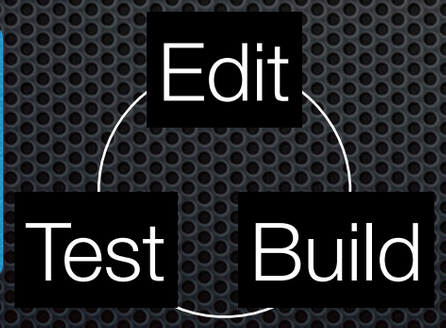
commit

Automated deployment pipeline (Build, test and deploy)

deployment

Production

Must be locally testable



Must be testable using automated tests

Architectural requirement: deployable

Handle >= 1 push / developer / day

Automated
Fast
Reliable
Incremental
Undoable

commit

Automated deployment pipeline (Build, test and deploy)

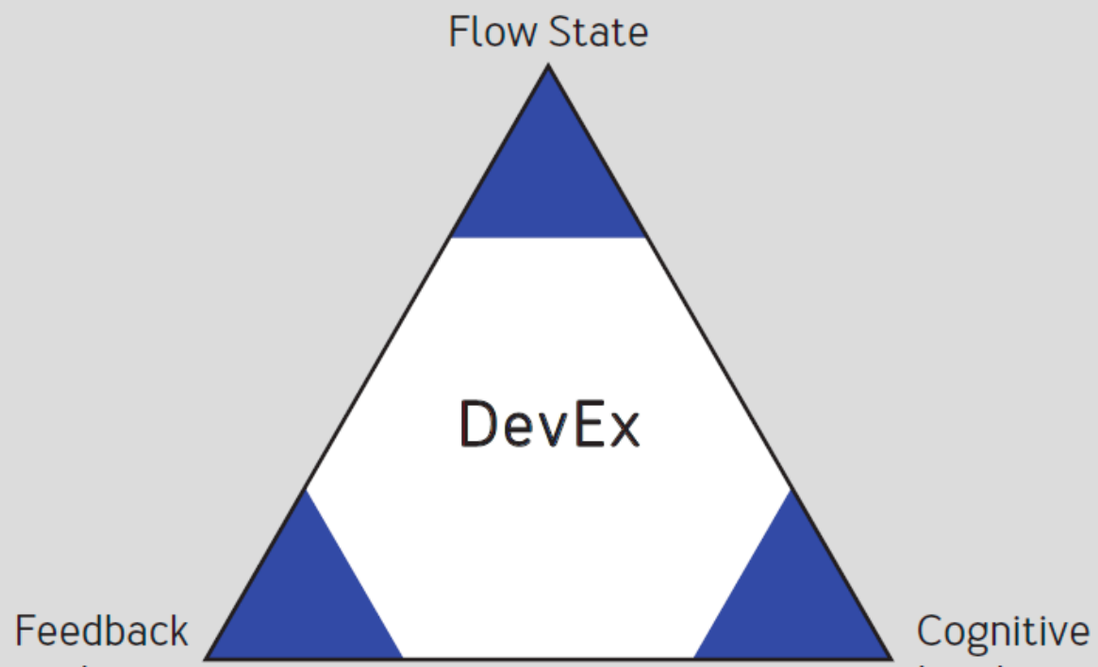
deployment

Production

Architecting for DevEx

Testability
Deployability
Observability

FIGURE 1: THREE CORE DIMENSIONS OF DEVELOPER EXPERIENCE



Simplicity
Loose coupling

https://queue.acm.org/detail.cfm?id=3595878

Architectural requirements distilled into dark energy and dark matter

Dark energy: an anti-gravity that's accelerating the expansion of the universe

Dark matter: an invisible matter that has a gravitational effect on stars and galaxies.

Simple components
Team autonomy
Fast deployment pipeline
…

Simple interactions
Prefer ACID or BASE
Minimize runtime coupling
…

Generated by system operations

https://www.nasa.gov/feature/goddard/2020/new-hubble-data-explains-missing-dark-matter

https://microservices.io/post/architecture/2023/03/26/dark-energy-dark-matter-force-descriptions.html

Dark energy and dark matter forces

Dark energy

Metaphor for

Simple components
Team autonomy
Fast deployment pipeline
Support multiple technology stacks
Segregate by characteristics

Repulsion

Service A

Subdomain A

«Aggregate»
X

Service B

Subdomain B

«Aggregate»
Y

Attraction

Simple interactions
Efficient interactions
Prefer ACID over BASE
Minimize runtime coupling
Minimize design time coupling

Metaphor for

Dark matter

https://microservices.io/post/architecture/2023/03/26/dark-energy-dark-matter-force-descriptions.html

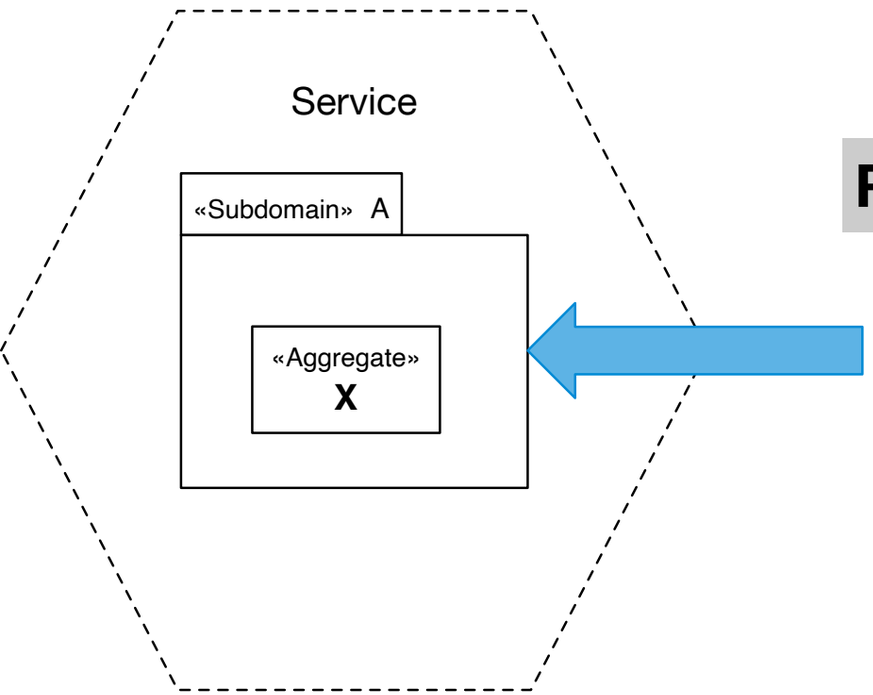
Agenda

- The need for fast flow
- A really brief overview of software architecture
- Architecting for fast flow
- Dark energy forces: encouraging decomposition
- Dark matter forces: resisting decomposition

A service exists to resolve

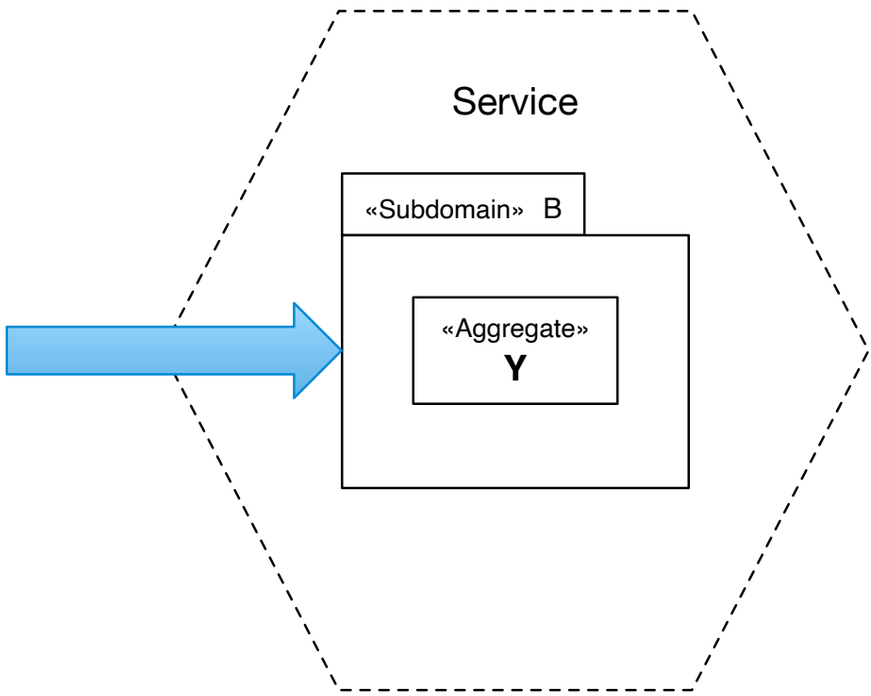
a dark energy force

Dark energy forces: reasons to use microservices



Repulsive dark energy forces

Simple components
Team autonomy
Fast deployment pipeline
Support multiple technology stacks
Segregate by characteristics

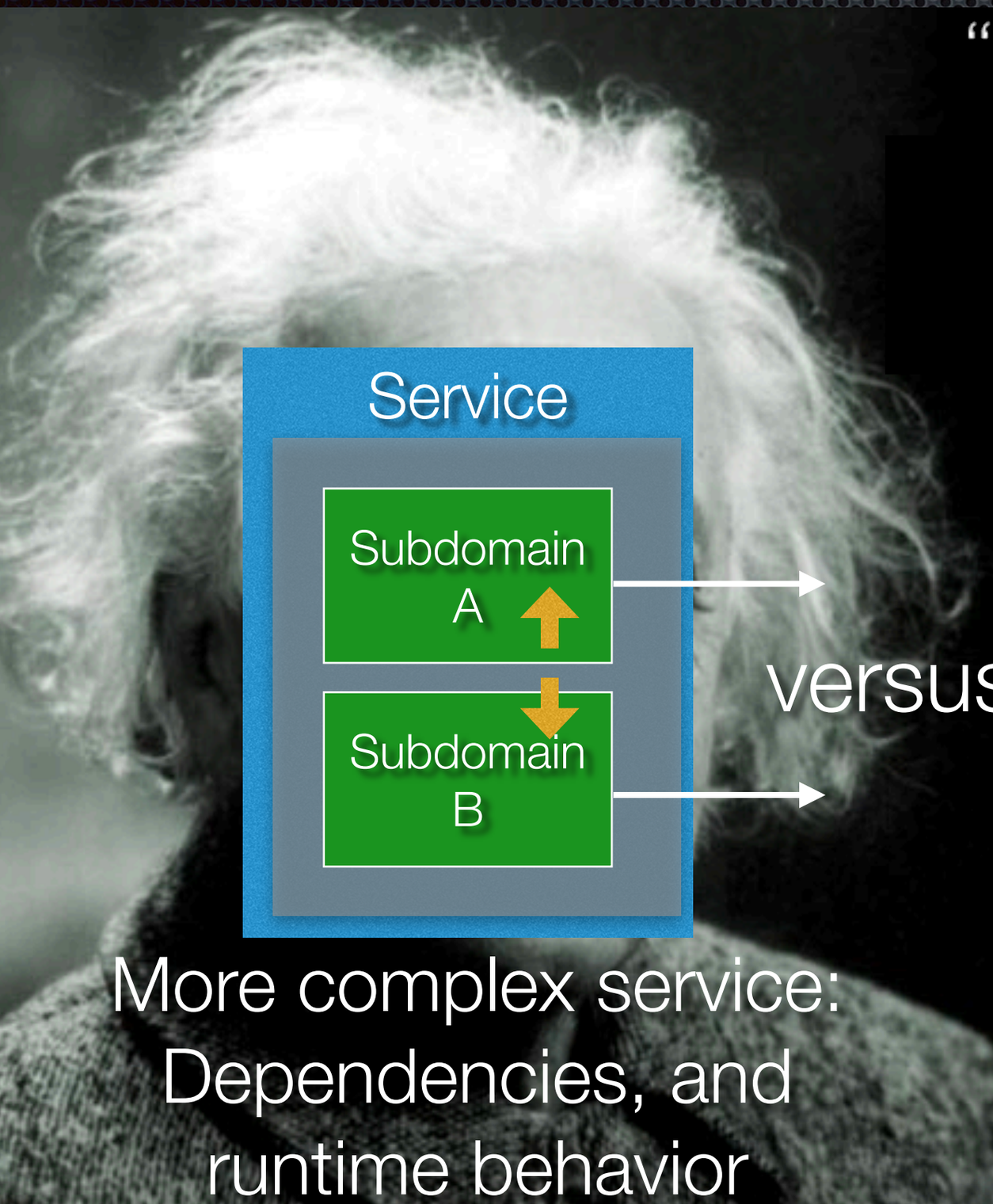


Characteristics:

- Resource requirements
- Regulations, e.g. SaMD/PCI
- Business criticality/tier
- Security, e.g. PII, …
- DDD core/supporting/generic
- ...

https://microservices.io/post/architecture/2023/03/26/dark-energy-dark-matter-force-descriptions.html

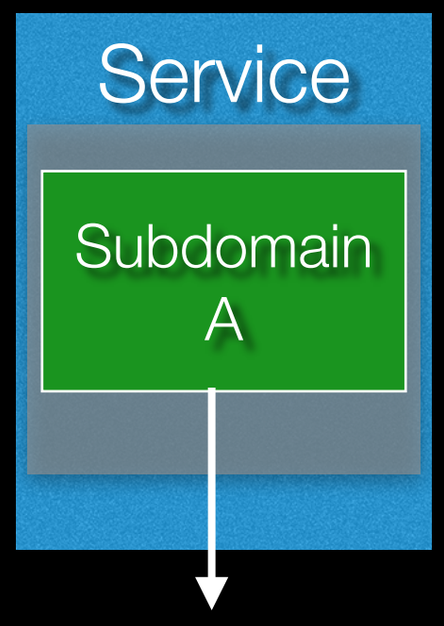
Simpler components/services

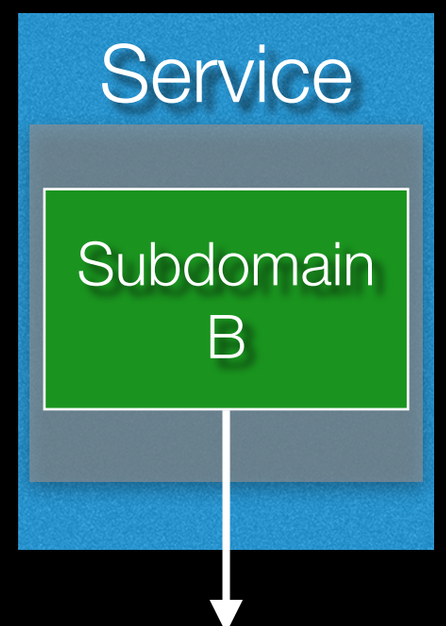


"Everything should be made as simple as possible. But not simpler."

Albert Einstein

versus





More complex service: Dependencies, and runtime behavior

Simpler services: easier to understand, develop, test, …

Team autonomy = service per team

Team A

Team B

Service

Subdomain A

Subdomain B

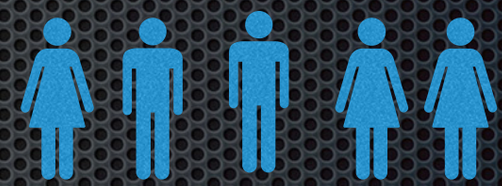
Coordination required

vs.

Team A

Service

Subdomain A

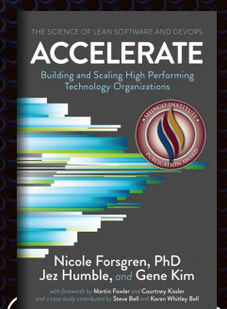


Team B

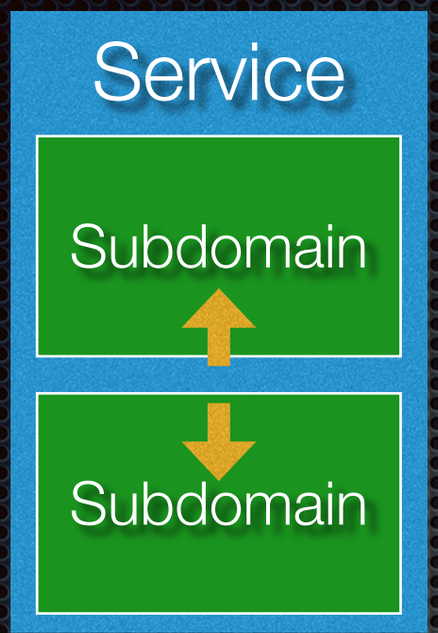
Service

Subdomain B

Build, test and deploy independently

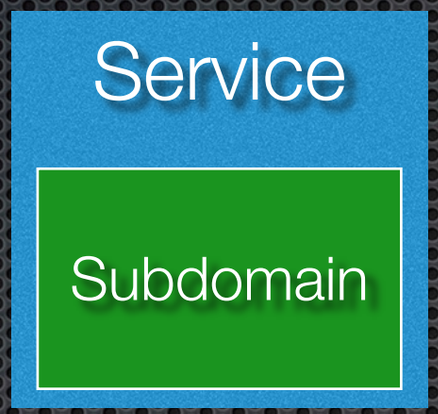


Fast deployment pipeline: short lead time/fast feedback



Longer lead time

More complex build*



Shorter lead time

Simpler build

What does a good feedback loop look like?

1. Engineer merges a change to master
2. Any user-visible changes hidden safely behind a feature flag
3. Which triggers CI to run tests and build an artifact
4. The artifact is then deployed or canaried to production (or deployed to staging and later promoted to production).

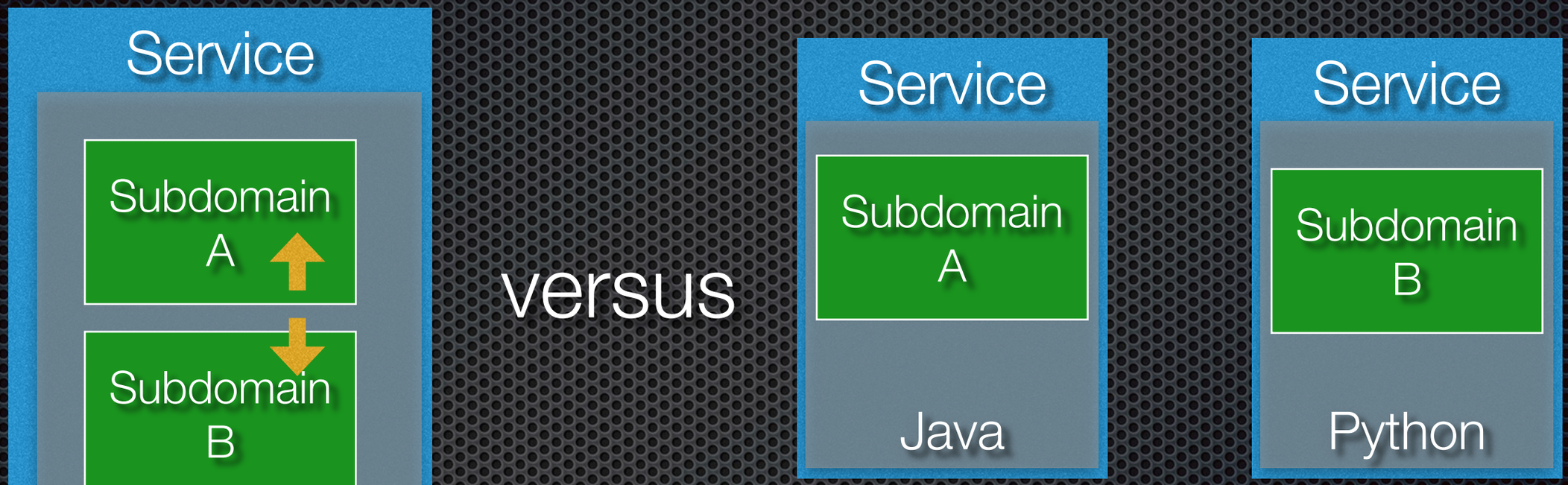
Time elapsed: **<15min**

Notice:
- Only one changeset by one engineer per build
- No manual gates. No manual QA or variable times
- Feature flags to decouple deploys from releases

@mipsytipsy
https://speakerdeck.com/charity/cd?slide=17

* Accelerate build/test:
- Merge queue
- Incremental testing through DIP and ISP
- Parallelization/clustered builds
- Selective test execution

Support multiple technology stacks: per-subdomain



Single technology stack

Larger upgrade task

Separate technology stacks

Right tool for the job

Smaller, incremental upgrades

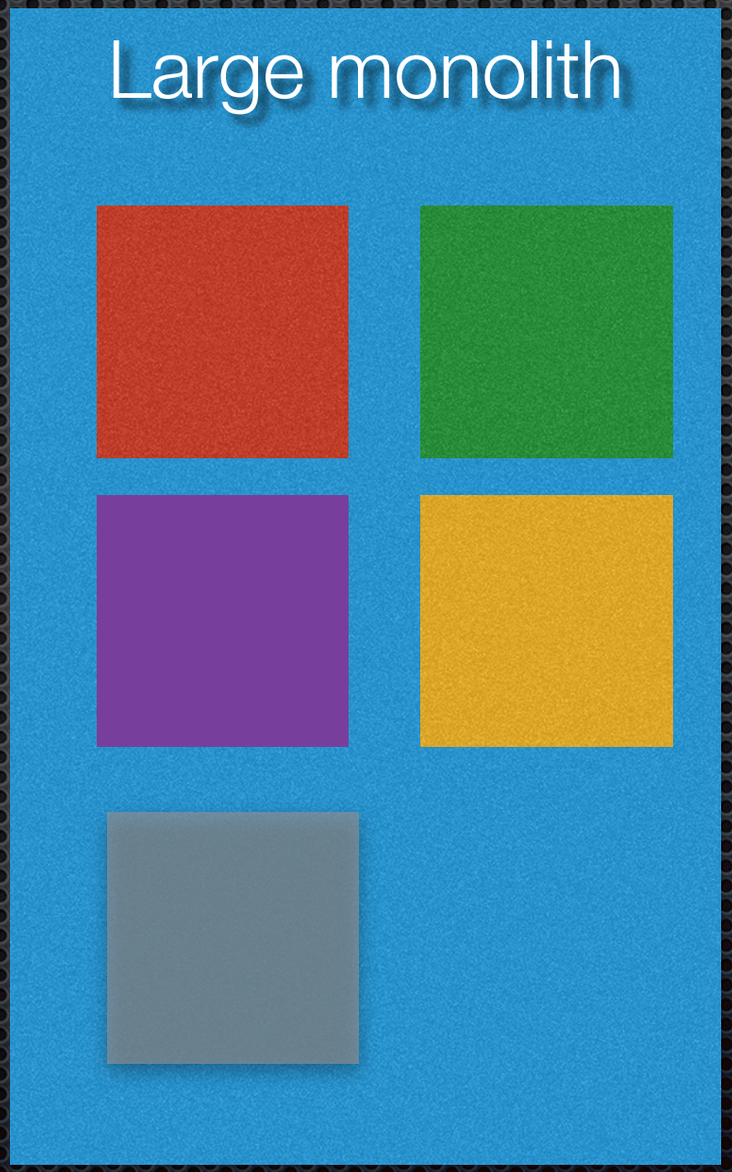
Separate subdomains by characteristics

Subdomain characteristic	Issue
Resource requirements	Cost-effective, scalability
Regulations, e.g. SaMD/ PCI	DevOps vs. Slower regulated process
Business criticality/tier	Maximize availability
Security, e.g. PII, …	Improve security
DDD core/supporting/ generic	Focus on being competitive

But **your** context determines relevance of dark energy forces to **your** application

Force: reason to use microservices	What determine relevance to **your** application
Team autonomy	∝ Number of teams
Fast deployment pipeline	Build time ∝ size/complexity of app Build frequency ∝ rate of Git pushes
Multiple technology stacks	Need multiple technologies?
Segregate by characteristics	
Business criticality	Have high availability subdomains?
Regulations	Regulated subdomains?
…	

IF you have this DevEx...

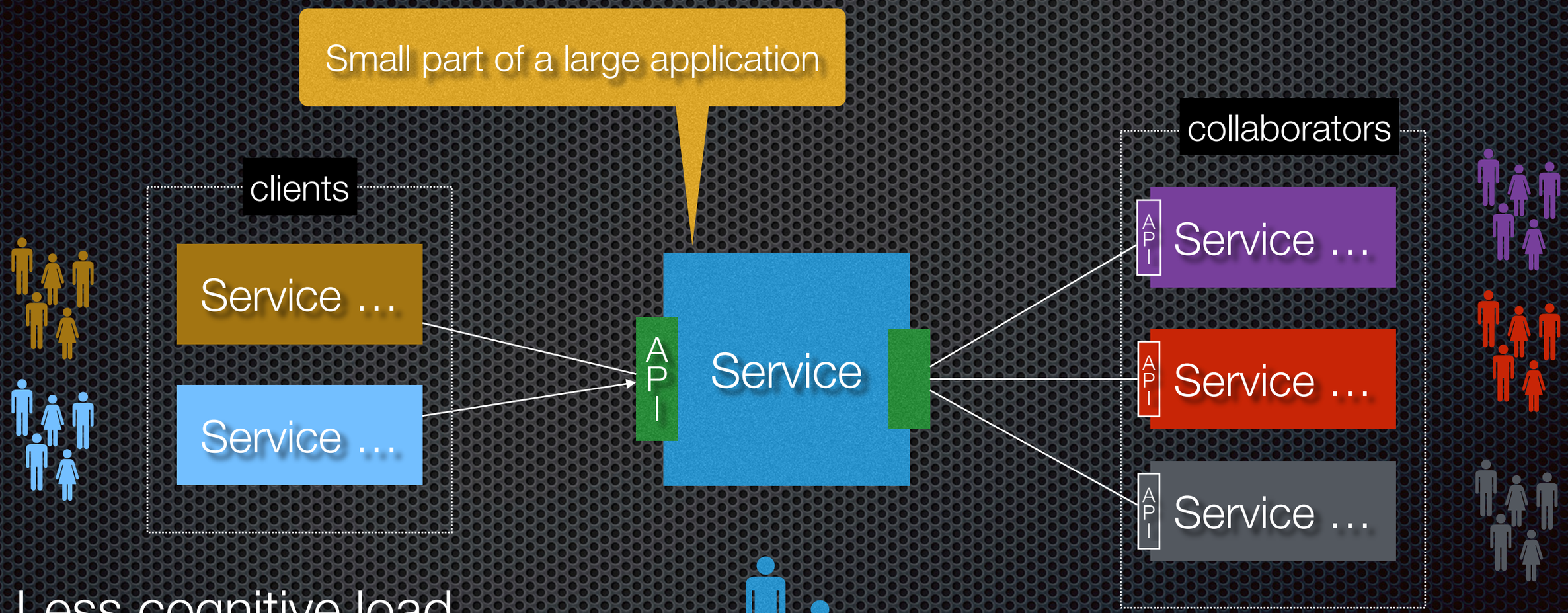


Simple (e.g. ACID txns) development BUT

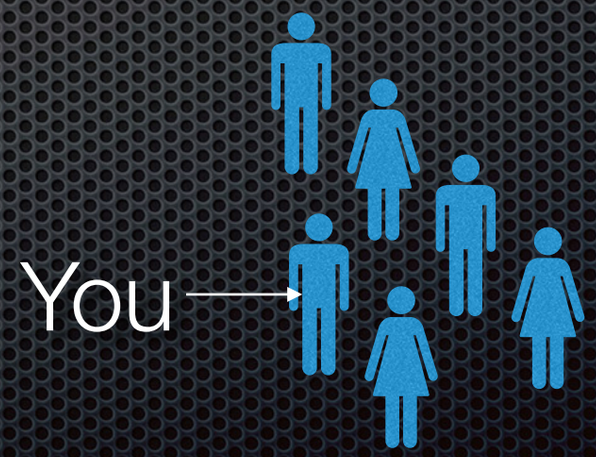
Single classpath, Large code base, Slow deployment pipeline, ...

- High cognitive load
- Less autonomy
- More meetings
- Slow feedback

...THEN adopt microservices to have this DevEx



- Less cognitive load
- More autonomy incl. technical
- Less meetings
- Faster feedback



Agenda

- The need for fast flow
- A really brief overview of software architecture
- Architecting for fast flow
- Dark energy forces: encouraging decomposition
- Dark matter forces: resisting decomposition

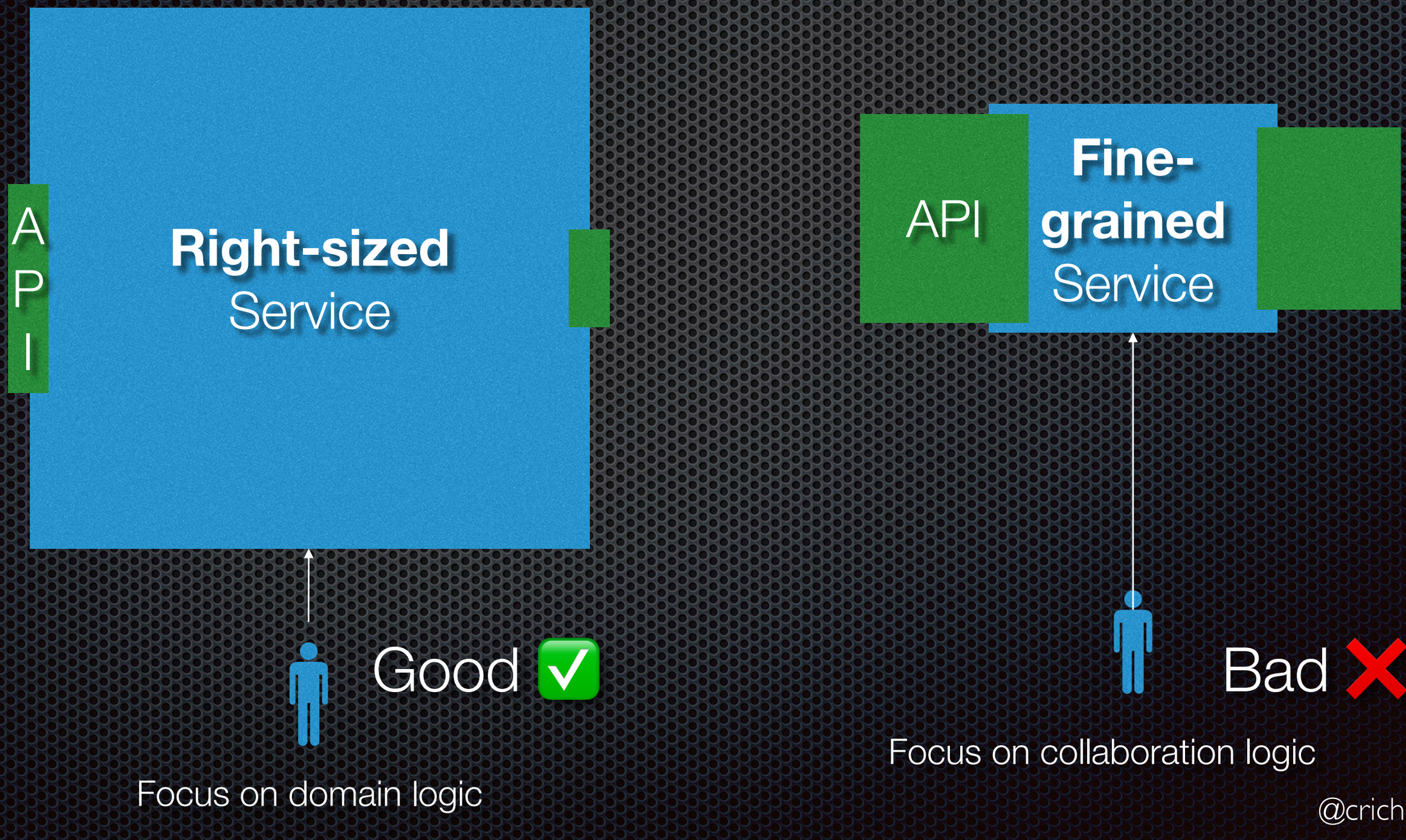
How many services?

- MICROservices != little services
- Excessively fine-grained, overly complex architectures are far too common
- A service should exist to solve a tangible problem

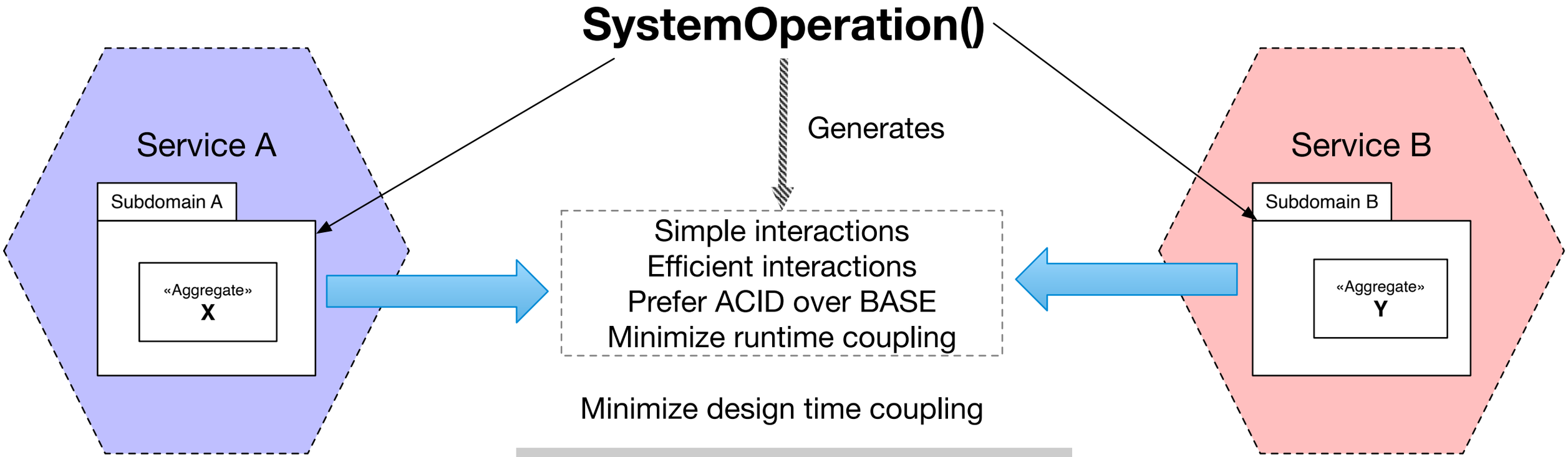
https://microservices.io/post/antipatterns/2019/05/21/antipattern-more-the-merrier.html

https://premium.microservices.io/a-service-should-exist-to-solve-a-problem/

Fine-grained architecture: risk of poor DevEx

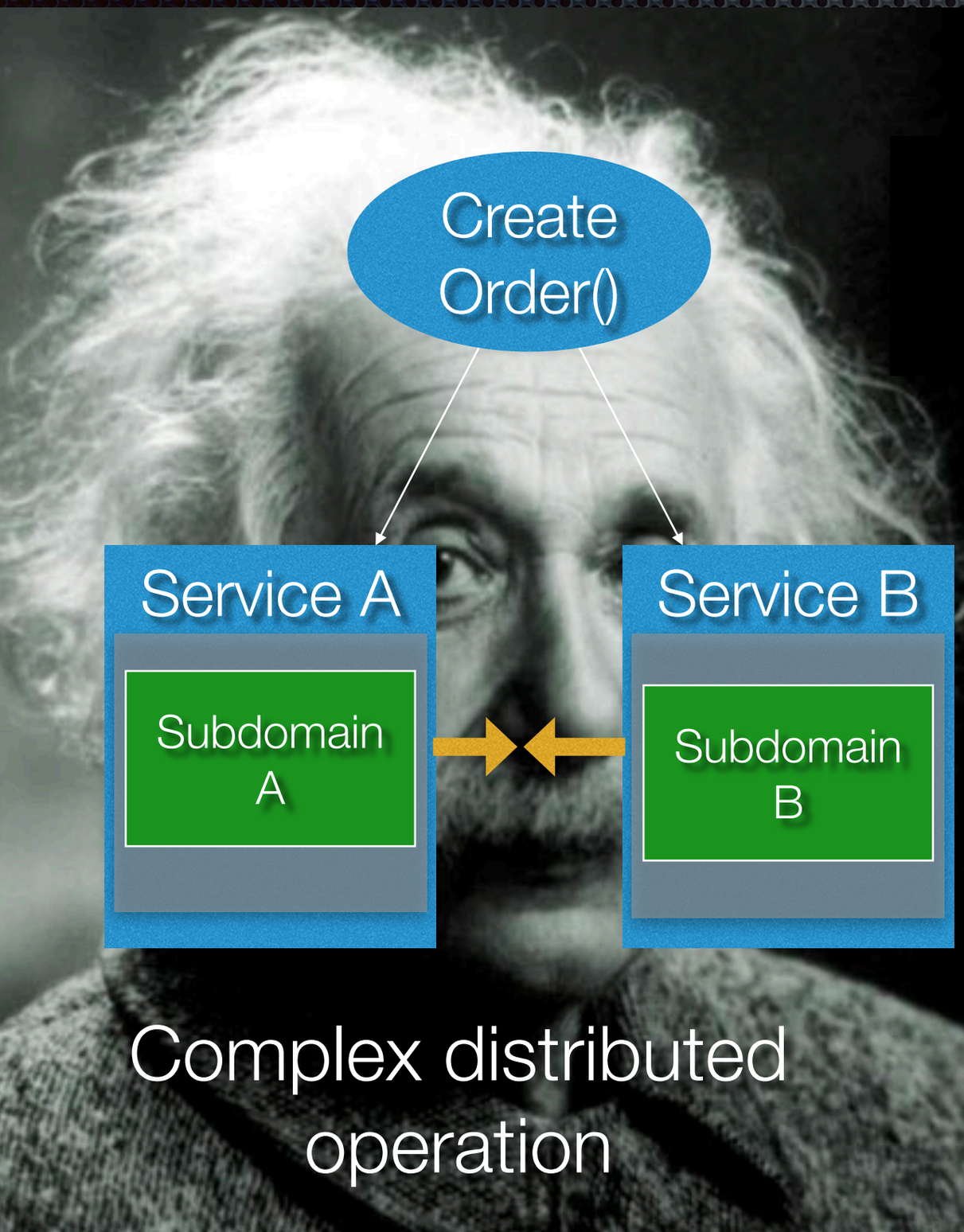


Dark matter forces: reasons NOT to distribute subdomains



https://microservices.io/post/architecture/2023/03/26/dark-energy-dark-matter-force-descriptions.html

Simple interactions

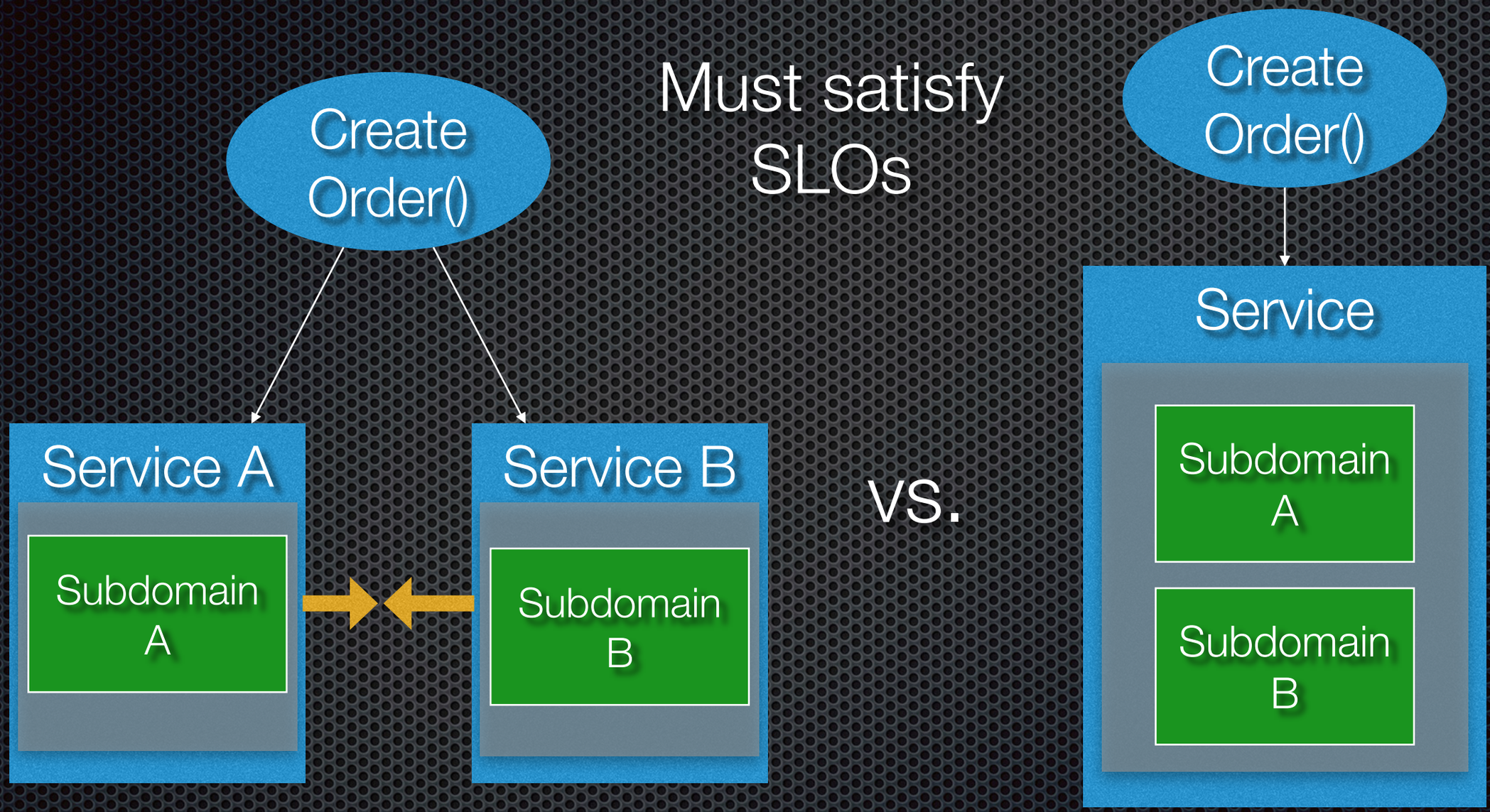


Complex distributed operation

vs.

Simple local operation: easier to understand, troubleshoot, …

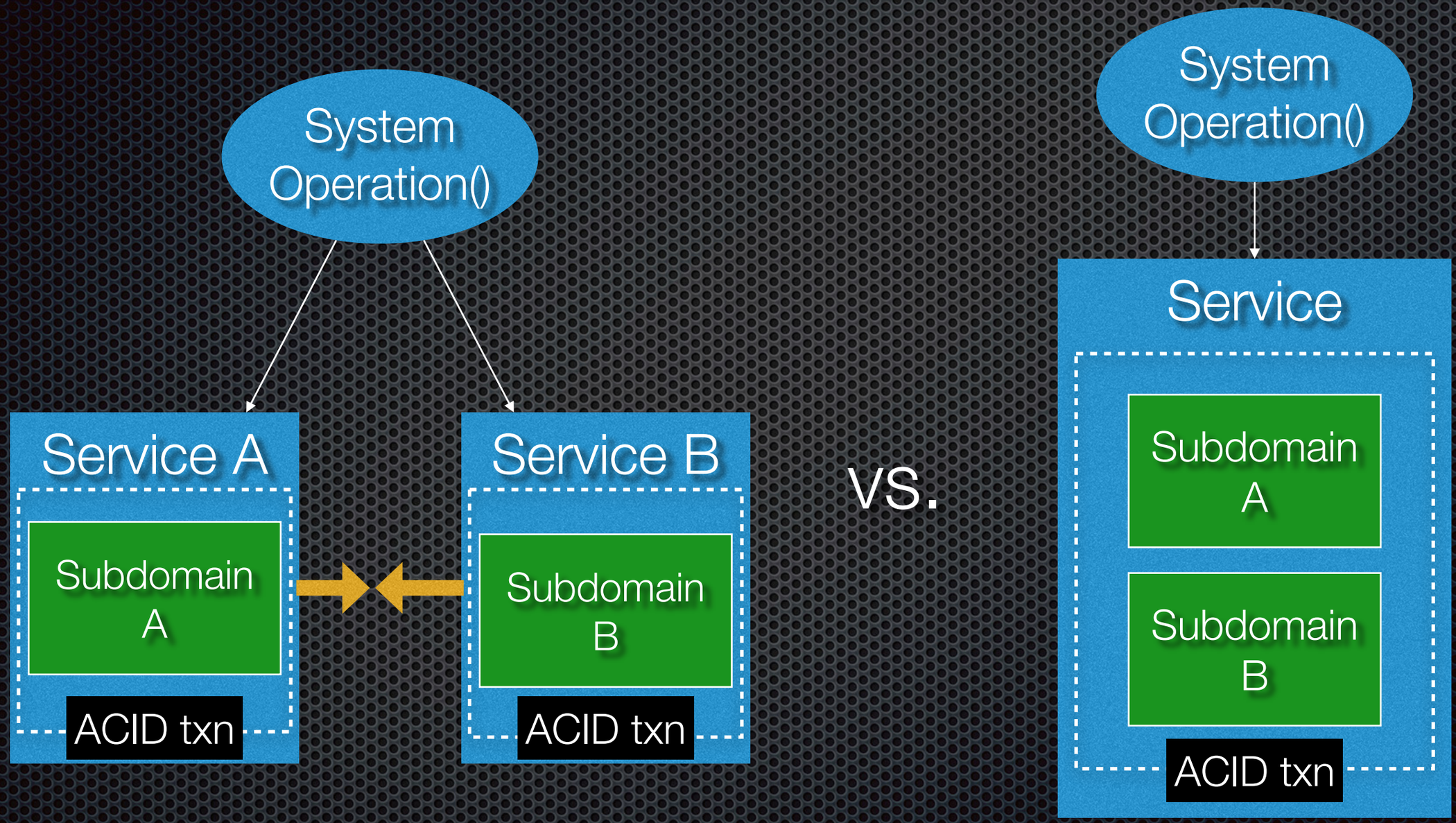
Efficient interactions



Network latency, limited bandwidth

In-memory, fast!

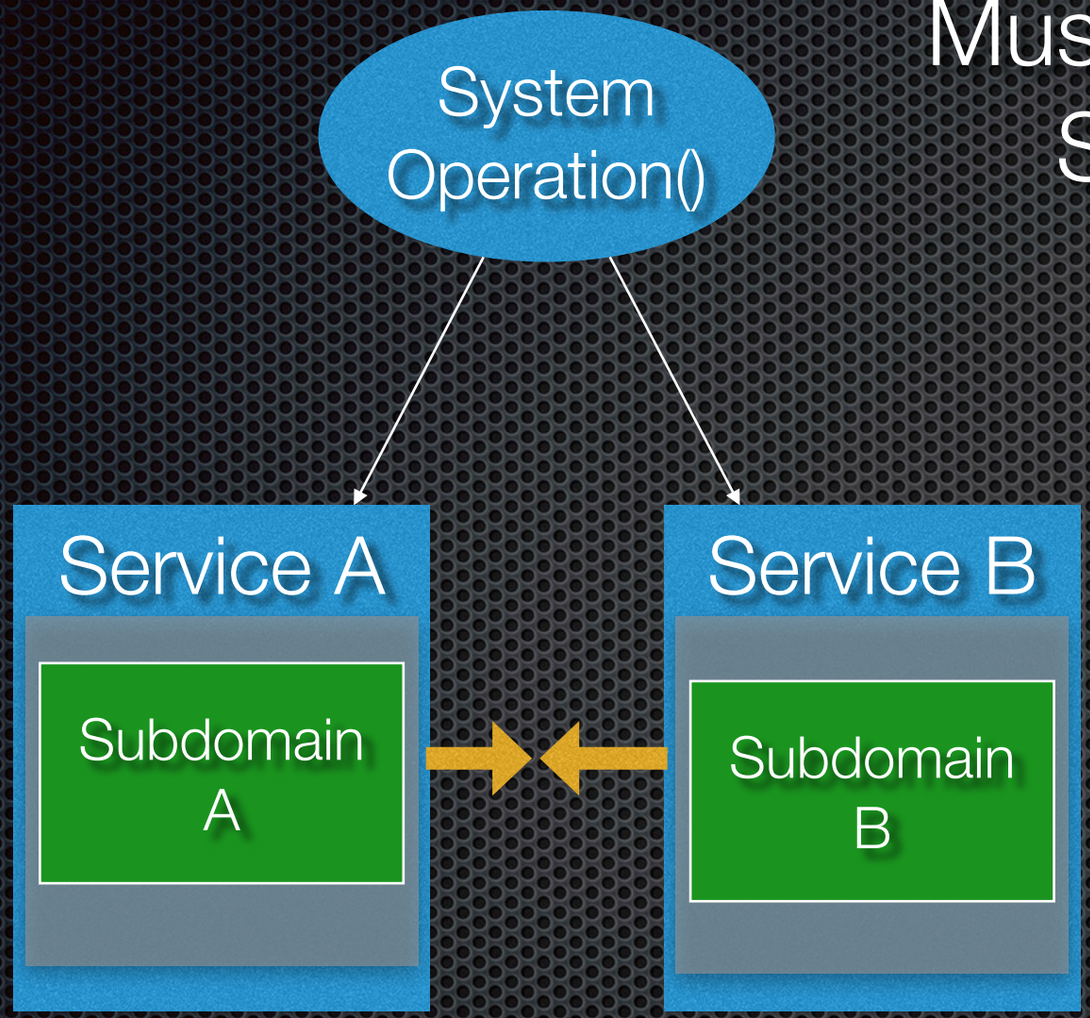
Prefer ACID over BASE



Distributed, eventually consistent transaction

Simple, local ACID transaction

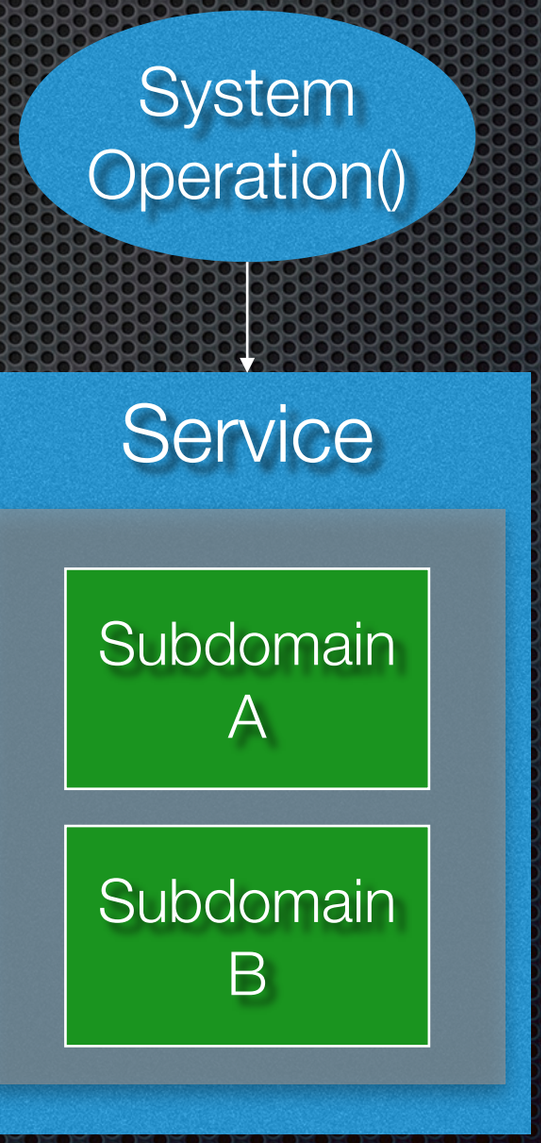
Minimize runtime coupling



Must satisfy SLOs

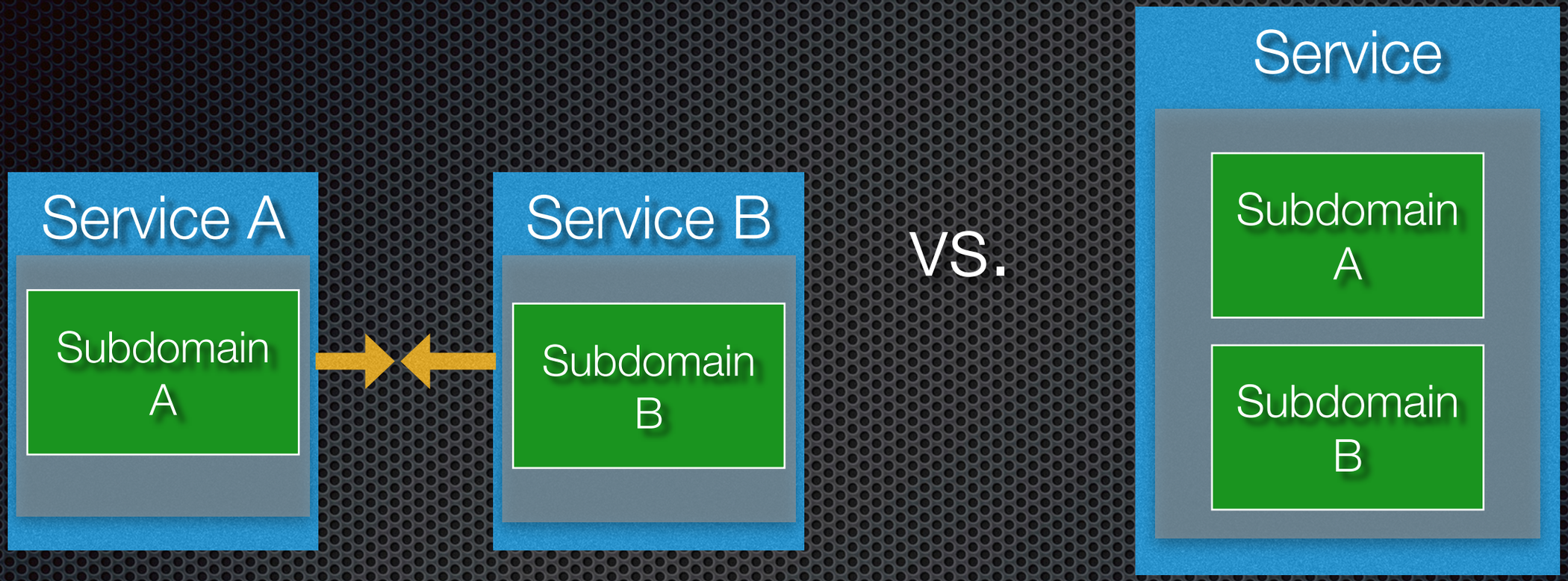
Risk of runtime coupling, e.g. not meeting SLOs

VS.



No runtime coupling: higher availability, lower latency

Minimize design-time coupling



For zero-downtime deployments:

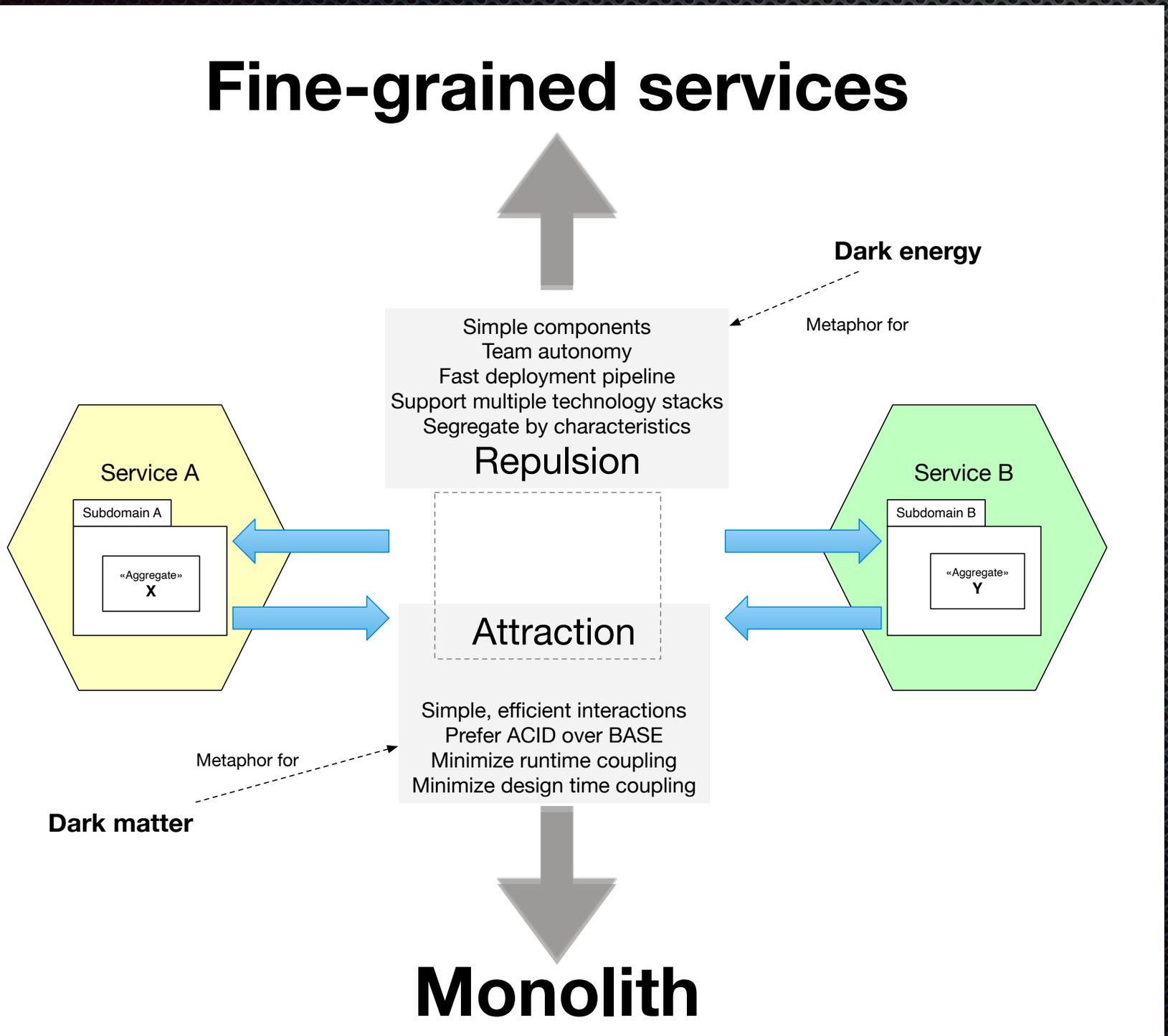
1. Change biz logic + **Add** new version of API
2. Migrate clients to new API
3. Remove old API

Simple commit

Your context determines relevance of dark matter forces to **your** application

Force: reason to NOT use microservices	What determine relevance to **your** application
Simple interactions	
Efficient interactions	Operation specification, SLO and design
Prefer ACID over BASE	
Minimize runtime coupling	
Minimize design coupling	Service and operation design

Designing an architecture = making trade-offs

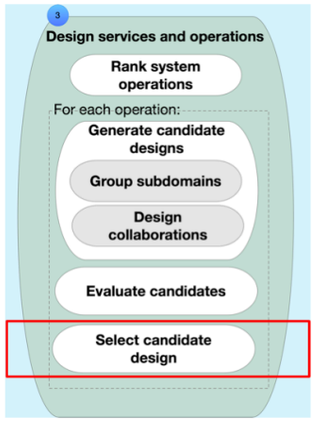


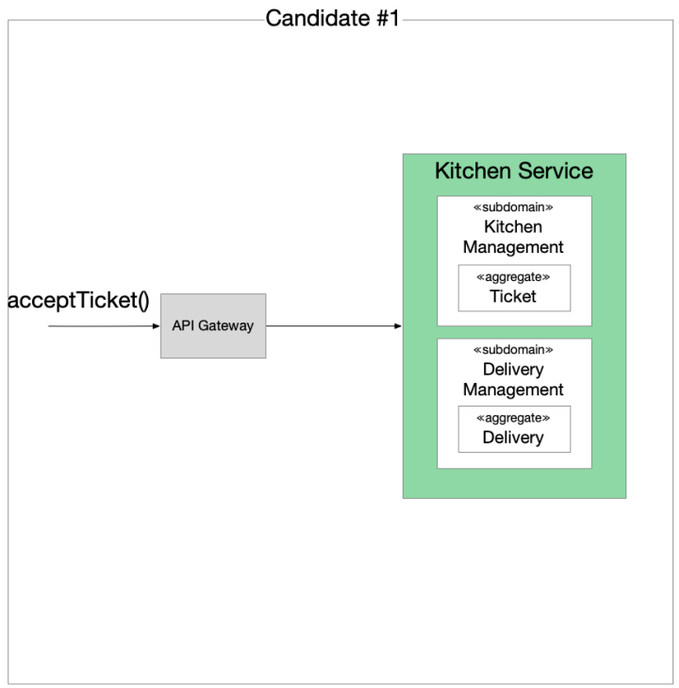
Conflicting forces

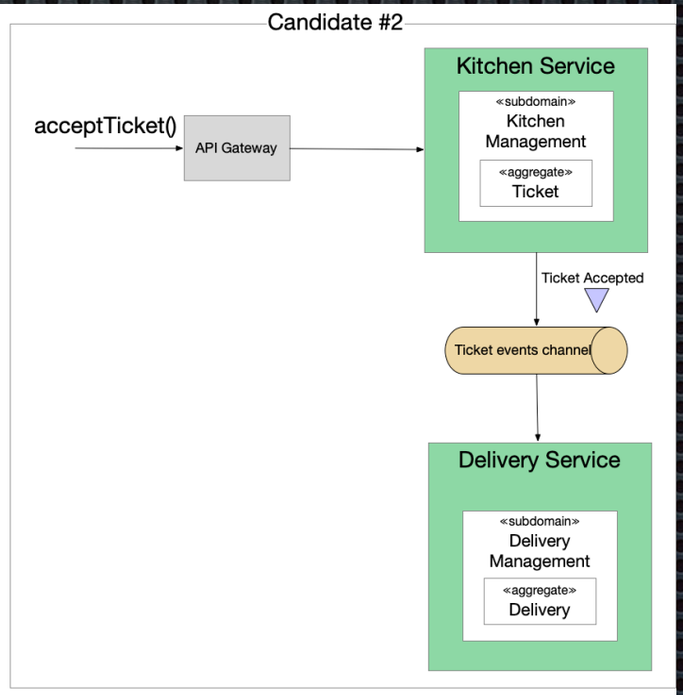
Therefore

- Some services/ operations will have an optimal design
- Others will be suboptimal but good enough

Evaluate and compare alternatives



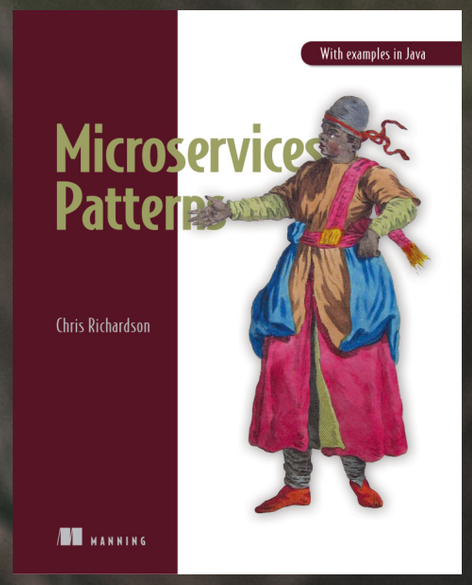




Repulsive dark energy forces		
Simple components	❌	✅
Team autonomy	❌	✅
Fast deployment pipeline	❌	✅
Support multiple technology stacks	✅	✅
Segregate by characteristics	✅	✅
Attractive dark matter forces		
Simple interactions	✅	✅
Efficient interactions	✅	✅
Prefer ACID over BASE	✅	✅
Minimize runtime coupling	✅	✅
Minimize design time coupling	✅	✅

@crichardson chris@chrisrichardson.net

Questions?



http://adopt.microservices.io